

MODERN_IO_REFERENCE

Core Definition

The C++20/C++23 additions to the I/O library, plus the small utility headers that round out `<cpp/iostream>`:

- `<syncstream>` (C++20) — thread-safe output without interleaving
- `<spanstream>` (C++23) — streams over a fixed external buffer, no allocation
- `<print>` (C++23) — `std::print` / `std::println`, formatted output done right
- `<format>` (C++20) — the formatting engine behind `<print>`
- `<iosfwd>` — forward declarations only, for cheap header includes
- **Removed/deprecated** — `<strstream>`, `std::codecvt`

#CPP #CPP20 #CPP23 #PRINT #FORMAT #SYNCSTREAM #SPANSTREAM #IOSFWD #THREADING

`<print>` — C++23

```
// =====
// FUNCTIONS — Target | Operation | Output
// =====
std::print(fmt, args...)           // format | Write to stdout | No trailing newline
std::print(stream, fmt, args...)   // FILE*/os | Write to a stream | ostream& or FILE*
std::println(fmt, args...)         // format | Write + '\n' | The common case
std::println(stream, fmt, args...) // + stream | Write + '\n' | 
std::println()                    // none | Just a newline | C++26
std::vprint_unicode(...)           // runtime | Type-erased, Unicode | For wrapper functions
std::vprint_nonunicode(...)        // runtime | Type-erased, raw | For wrapper functions

// Format strings are checked at COMPILE TIME. A wrong placeholder count or an
// unformattable type is a compile error, not a runtime surprise.
```

```
#include <print>

std::println("Hello, {}!", name);
std::println("{:>10} {:>8.2f}", item, price);
std::print("no newline");
std::println(stderr, "error: {}", msg); // To a FILE*
std::println(std::cout, "to a stream: {}", x); // To an ostream
```

`<format>` — C++20

```
// =====
// FUNCTIONS
// =====
std::format(fmt, args...)           // format | Build a string | Returns std::string
std::format(loc, fmt, args...)       // + locale | Locale-aware | Returns std::string
std::format_to(out, fmt, args...)    // iterator | Write through iterator | Returns the iterator
std::format_to_n(out, n, fmt, ..)    // + limit | Bounded write | Returns {out, size}
std::formatted_size(fmt, args...)    // format | Required length | Returns size_t
std::vformat(fmt, args)              // runtime | Runtime format string | Returns std::string
std::make_format_args(args...)        // args | Type-erase arguments | For vformat
// Takes LVALUE references — make_format_args(1, 2)
// does not compile. Name the values first.
```

```

std::runtime_format(str)           // C++26      | Mark a string as runtime-checked

// =====
// FORMAT SPEC: {[index]::[[fill]align][sign][#][0][width][.precision][L][type]]}
// =====
{} // Next argument, default format
{0} {1} {0} // Explicit indices – reuse an argument
{{ }} // Literal braces

// ALIGN          SIGN          OTHER
< left          + always      # alternate form (0x, 0b, force '.')
> right         - negative only 0 zero-pad
^ center        (space) space for + L locale-aware separators

// WIDTH / PRECISION
{:10}          width 10          {:.3}          precision 3
{:{}}          width from an argument {:.{}}          precision from an argument

// TYPES – integers          TYPES – floats          TYPES – other
b binary          a/A hex float          s string
B binary, 0B prefix e/E scientific        c character
d decimal (default) f/F fixed            p pointer
o octal          g/G general (default) ? debug/escaped (C++23)
x hex lower      (no type = shortest round-trip)
X hex upper
c as a character

// EXAMPLES
std::format("{:>8}", "hi") // "      hi"
std::format("{:*^11}", "mid") // "****mid****"
std::format("{:08.3f}", 3.14159) // "0003.142"
std::format("{:#010b}", 42) // "0b00101010"
std::format("{:+}", 42) // "+42"
std::format("{:L}", 1234567) // "1,234,567" (with a suitable locale)
std::format("{:{}.{}f}", 3.14159, 10, 2) // "      3.14"
std::format("{1} {0}", "a", "b") // "b a"
std::format("{:?}", "a\nb") // "\a\nb" (C++23 debug format)

// =====
// CONTAINERS & RANGES (C++23)
// =====
std::format("{} ", std::vector{1,2,3}) // "[1, 2, 3]"
std::format("{} ", std::map<int,int>{{1,2}}) // "{1: 2}"
std::format("{} ", std::pair{1, "a"}) // "(1, \"a\")"
std::format("{::>4}", std::vector{1,2}) // "[ 1, 2]" – spec after :: applies per element
std::format("{:n}", std::vector{1,2,3}) // "1, 2, 3" – no brackets

// =====
// CUSTOM TYPE SUPPORT
// =====
struct Point { int x, y; };

template <>
struct std::formatter<Point> {
    constexpr auto parse(std::format_parse_context& ctx) { return ctx.begin(); }

    auto format(const Point& p, std::format_context& ctx) const {
        return std::format_to(ctx.out(), "({}, {})", p.x, p.y);
    }
};

// Simplest path – delegate to an existing formatter:
template <>
struct std::formatter<Point> : std::formatter<std::string> {
    auto format(const Point& p, std::format_context& ctx) const {
        return std::formatter<std::string>::format(
            std::format("({}, {})", p.x, p.y), ctx);
    }
};

// This inherits the whole spec grammar – {:>20} on a Point then works.

```

<syncstream> — C++20

The problem it solves: two threads writing to `std::cout` produce correctly-typed but arbitrarily interleaved characters. `osyncstream` accumulates into a private buffer and transfers it to the destination atomically when destroyed (or on `emit()`).

```
// =====
// CLASSES & FUNCTIONS
// =====
std::osyncstream os(dest)           // ostream/buf | Synchronized wrapper | Emits on destruction
std::syncbuf                        // The underlying buffer type
os.emit()                           // none      | Transfer NOW      | Returns bool
os.get_wrapped()                    // none      | The destination buf | Returns streambuf*
os.rdbuf()                          // none      | The syncbuf        | Returns syncbuf*
std::emit_on_flush                  // manipulator | Emit on each flush
std::noemit_on_flush                // manipulator | Don't (DEFAULT)
std::flush_emit                     // manipulator | Flush AND emit
```

```
#include <syncstream>
#include <iostream>
#include <thread>

void worker(int id) {
    // Everything written here lands as one uninterrupted block
    std::osyncstream out(std::cout)
        << "thread " << id << " starting\n"
        << "thread " << id << " done\n";
} // Temporary destroyed → atomic emit

void namedForm(int id) {
    std::osyncstream out(std::cout);
    out << "step 1 of " << id << '\n';
    out << "step 2 of " << id << '\n';
    out.emit(); // Explicit early transfer
    out << "step 3\n";
} // Remainder emitted here

int main() {
    std::vector<std::jthread> ts;
    for (int i = 0; i < 4; ++i) ts.emplace_back(worker, i);
}
```

Without `osyncstream`, `std::cout` writes are individually thread-safe (no data race, no corruption) but freely interleaved between insertions. `osyncstream` is what makes a multi-line message stay together. It does **not** order threads — only groups each thread's output.

<spanstream> — C++23

Streams over memory you already own. No allocation, no `std::string` — useful in embedded, real-time, and hot-path code, and as the modern replacement for the removed `<strstream>`.

```
// =====
// CLASSES
// =====
std::spanbuf           // The buffer over a span
std::ispanstream       // Input from a span
std::ospanstream       // Output into a span
std::spanstream        // Both
// Wide variants: wspanbuf, wspanstream, wospanstream, wspanstream

os.span()              // none      | The WRITTEN portion | Returns span<char>
os.span(sp)            // span    | Rebind to new memory | Returns void
```

```
#include <spanstream>
#include <span>

char buffer[128];
std::ospanstream os(std::span<char>{buffer});
os << "id=" << 42 << " ok";
std::span<char> written = os.span();           // Exactly the bytes produced
// std::string_view sv(written.data(), written.size());

// Parsing without copying:
const char* data = "10 20 30";
std::ispanstream is(std::span<const char>{data, std::strlen(data)});
int a, b, c;
is >> a >> b >> c;
```

Overflow does not grow the buffer — it sets `failbit`. Check the stream after writing.

`<iosfwd>`

Forward declarations for every stream type, and nothing else. Include it in *headers* that only mention stream types in signatures; include the real header in the `.cpp`.

```
// my_class.hpp
#include <iosfwd>

class MyClass {
public:
    void print(std::ostream& os) const;    // Only a reference – a declaration suffices
};
std::ostream& operator<<(std::ostream&, const MyClass&);

// my_class.cpp
#include "my_class.hpp"
#include <ostream>                       // The definition, only where it's used
```

Declares: `basic_ios`, `basic_streambuf`, `basic_istream`, `basic_ostream`, `basic_iostream`, `basic_stringbuf`/`stringstream` family, `basic_filebuf`/`fstream` family, `basic_syncbuf`/`osyncstream`, `basic_spanbuf`/`spanstream` family, `fpos`, `streampos`, and all the `char/wchar_t` aliases.

REMOVED AND DEPRECATED

```
<sstream> // Deprecated C++98, REMOVED C++26
// sstreambuf, istream, ostream, stringstream
// Replacement: <sstream>, or <spanstream> for fixed buffers

std::codecvt<char16_t, char, mbstate_t> // Deprecated C++20
std::wstring_convert / wbuffer_convert // Deprecated C++17
// No standard replacement; use ICU, iconv, or
// a library like simdutf / utfcpp for encoding conversion

std::gets // <stdio> – REMOVED in C++14 (unbounded overflow)
```

CHOOSING AN OUTPUT TOOL

Situation	Use
Printing to the console, C++23 available	<code>std::println("{} ", x)</code>
Building a string, C++20 available	<code>std::format("{} ", x)</code>
Writing into an existing buffer	<code>std::format_to_n / ospanstream</code>
Number $\leftrightarrow$ string, fastest, locale-free	<code>std::to_chars / from_chars</code>
Console output, pre-C++20	<code>std::cout << x</code>
Multi-threaded console output	<code>std::osyncstream(std::cout) << ...</code>
Writing to a file	<code>std::ofstream + << (or std::print(file, ...))</code>
Interfacing with C code	<code>std::snprintf / FILE*</code>
Custom types through a generic API	<code>operator<< (works everywhere)</code>

IMPORTANT CONCEPTS

`std::print` Is Not Just `printf` with Braces

The format string is a compile-time-checked `constexpr` parameter. `std::println("{} {}", 1)` fails to compile. A type with no `formatter` specialization fails to compile. There are no `varargs` and no promotion surprises. To use a format string only known at runtime, go through `std::vformat` (or C++26's `std::runtime_format`).

`std::print` Handles Unicode on Windows

On a Windows console, `std::print` writes UTF-8 correctly where `printf` and `cout` historically mangled it — it detects a console destination and uses the wide API. This alone is a reason to prefer it for user-facing output.

`make_format_args` Needs Named Values

```
std::vformat("{} {}", std::make_format_args(1, 2)); // Does NOT compile
int a = 1, b = 2;
std::vformat("{} {}", std::make_format_args(a, b)); // Correct
```

It binds lvalue references, deliberately — the original forwarding-reference signature made it easy to store a dangling reference to a temporary. Bind the arguments to named variables that outlive the `vformat` call.

`format` Allocates; `format_to` Need Not

`std::format` returns a `std::string`. In a hot path, `std::format_to` with a `back_insert_iterator` into a reused buffer, or `format_to_n` into a stack array, avoids the allocation.

`osyncstream` Groups, It Does Not Order

Each thread's block arrives intact, but the order of blocks between threads is whatever the scheduler produces. If you need ordering, you need a queue and a single writer, not a `syncstream`.

An `osyncstream` Emits on Destruction

The temporary form `std::osyncstream(std::cout) << ...;` emits at the end of the full expression — which is what makes it a one-liner. A named object holds its content until it goes out of scope, so keep its lifetime tight or call `emit()` explicitly.

`spanstream` Never Grows

It is a fixed window into memory you provided. Writing past the end sets `failbit` rather than reallocating. Check the stream state, or size the buffer with `std::formatted_size` first.

`<iosfwd>` Is About Build Times

`<iostream>` pulls in a large amount of the standard library, including static initialization for the global stream objects. A header that only declares `std::ostream&` parameters should include `<iosfwd>` instead — the difference across a large project is measurable.

Including `<iostream>` Has a Side Effect

It declares a static `std::ios_base::Init` object that constructs `cin / cout / cerr / clog`. That is why including `<iostream>` in a translation unit that never uses it still costs something at startup.

MIGRATION QUICK TABLE

Old	New
<code>printf("%d\n", x)</code>	<code>std::println!("{}", x)</code>
<code>sprintf(buf, "%d", x)</code>	<code>std::format_to_n(buf, n, "{}", x)</code>
<code>ostringstream ss; ss << a << b;</code>	<code>std::format("{}{}", a, b)</code>
<code>cout << setw(8) << fixed << setprecision(2) << v</code>	<code>std::print("{:8.2f}", v)</code>
<code>stringstream</code> for number parsing	<code>std::from_chars (<charconv>)</code>
<code>ostream</code> over a <code>char[]</code>	<code>std::ospanstream</code> over a <code>span</code>
mutex around <code>cout</code> in threads	<code>std::osyncstream(std::cout)</code>

BEST PRACTICES

1. Prefer `std::println` for console output when C++23 is available — checked, Unicode-correct, concise
2. Prefer `std::format` over `ostringstream` for building strings
3. Use `format_to` / `format_to_n` when you want to avoid the allocation
4. Specialize `std::formatter` by inheriting from an existing one to get the whole spec grammar for free
5. Wrap every multi-line threaded write in an `osyncstream`
6. Keep named `osyncstream` lifetimes short, or call `emit()` deliberately
7. Check `failbit` after writing to an `ospanstream` — overflow is silent otherwise
8. Include `<iosfwd>` in headers, the real stream header in sources
9. Use `std::vformat` for runtime format strings — never build one and pass it to `format`
10. Don't reach for `<sstream>` — it is gone; `<spanstream>` is the replacement

11. **Keep `operator<<` for your types** even when using `std::print` — it keeps them usable with every stream-based API; add a `formatter` alongside it

RELATED HEADERS

```
#include <print>           // C++23: print, println
#include <format>           // C++20: format, format_to, formatter
#include <syncstream>       // C++20: osyncstream, syncbuf
#include <spanstream>       // C++23: ispanstream, ospanstream, spanbuf
#include <iosfwd>           // Forward declarations for all stream types
#include <charconv>         // C++17: to_chars, from_chars
#include <span>             // C++20: the buffer type spanstream takes
#include <iostream>         // The classic streams
#include <thread>           // What makes syncstream necessary
```

EXTERNAL RESOURCES

- **<print>**: <https://en.cppreference.com/w/cpp/header/print>
- **Format specification**: <https://en.cppreference.com/w/cpp/utility/format/spec>
- **std::osyncstream**: https://en.cppreference.com/w/cpp/io/basic_osyncstream
- **std::spanstream**: https://en.cppreference.com/w/cpp/io/basic_spanstream
- **<iosfwd>**: <https://en.cppreference.com/w/cpp/header/iosfwd>

Standard: C++20 (`format`, `syncstream`) / C++23 (`print`, `spanstream`, range formatting) / C++26 (`runtime_format`, `println()`) **Compiler support:** `<format>` needs GCC 13+, Clang 17+, MSVC 19.29+. `<print>` needs GCC 14+, MSVC 19.37+; Clang routes through libc++ 18+. **Last Updated:** September 2026